

a GUI package called *smart-notifier*, which just gives warnings.

## Copying, backup and version control

For copying directories locally or across a network, *rsync* is the tool. It can copy a whole directory, leaving the files that are already present and new on the destination, so that time and bandwidth can be saved. A sysadmin can use this to clone something or take a backup while Web developers can use this to update their websites. However, one can easily commit mistakes with it, so read more about it and experiment with it before testing it on important files.

Sometimes one needs a bit-by-bit copy of a disk, which is called a disk image. It is useful for safe data recovery, copying OS disks, backup, etc. While the classic UNIX commands *cp* and *dd* are capable of doing this, there are other user-friendly options like Clonezilla and GNOME Disks.

If you are one of those who believe copying doesn't make for a good backup scheme, then go for Déjà Dup, which offers features like incremental backup and encryption. Déjà Dup relies on Duplicity, which in turn relies on *rsync*.

If you are a software developer or the manager of a project with text files, version control systems like Git are great to keep track of the changes and roll back when needed. However, they do not replace backups in case of disk failures.

**Comparison and verification:** *diff* helps to compare two files. But if you want to verify the integrity of a downloaded file by comparing it against the MD5 checksum that the publisher has given, use *md5sum* to calculate the checksum of the file you have.

*diff* has limited support for comparing two directories, but *rsync* will be more useful and practical for this purpose.

## Archiving, compression, splitting and merging

Archiving is the process of encapsulating multiple files and directories into a single file while compression saves space. We usually use both while zipping. File managers usually provide options to create and manage them, but it would be useful to get familiar with the dedicated tools.

One can use commands like *tar*, *7z*, *zip*, etc, to archive and compress files. Some provide encryption support also. For example, password support in *7z* is considered good. GUI tools like GNOME Archive Manager (file-roller) help do these interactively. *archivemount* helps mount an archive and handle it just like a folder.

The *split* command helps to split a file into smaller chunks, which can later be joined using *cat*.

**Encryption:** LUKS is the *de facto* standard specification for full-disk encryption in GNU/Linux. This feature is listed in popular partitioning/formatting tools. One can also use the command line tool *cryptsetup*.

If you want to encrypt a single file or an archive, the options include *gpg* or simply the password feature supported by archiving formats like *7z*.

## RAID management

RAID can offer data protection and speed, based on the configurations we choose. In GNU/Linux, software RAID is created and managed using *mdadm*.

## Repair and recovery

*fsck* is the standard tool to check and repair a Linux file system. This is useful in cases like unexpected power-offs. However, if one wants to recover deleted partitions and files, perhaps the best package is *testdisk* (which *photorec* is part of).

**⚠ Cautionary note:** Be warned that data recovery can be dangerous. Seek expert support if you are not sure about what you are doing.

## Shredding and wiping

When one deletes a file, the system simply removes the link to it, and that's why it's a quick process. Although the actual contents of the deleted file will get overwritten as new files come, until then, the file can be recovered. To prevent this, one has to shred a file before deleting it, and the command is, well, *shred*.

*srm* (package: *secure-delete*) will do both shredding and deletion. There is also *wipe*, which helps to wipe the contents of a whole partition or disk.

## Remote files

Popular file managers like Nautilus support network based file management, although sometimes one will need to install some supporting packages. There are different ways to handle remote files, and personally, I find SSH-based ones to be the most reliable on GNU/Linux. SFTP (not FTPS) is an SSH-based file transfer protocol, which benefits from the encryption and authentication features of SSH. Simply press *Ctrl+L* (to open the location bar) in Nautilus or a compatible one, and give the address *sftp://IPADDRESS/OPTIONAL\_PATH* to access a remote directory.

There is a magical tool called *sshfs* which helps mount a remote directory so that it can be used like a local one. This is very useful in command-line-only situations.

Seems like our list itself can fill a disk... so it's time to stop. Take this listing just as a starting point and start exploring! **END** 

### By: Nandakumar Edamana

The author is a developer and technology writer who primarily focuses on software freedom, privacy and other ethical issues. He has authored a couple of books and several articles for mainstream publishers. A GNU/Linux user for more than ten years, he releases his software products under free software licences, which are available on [nandakumar.co.in](http://nandakumar.co.in).